



Understand the storage class

This lab must be done on Unix-based OS

1. Static Storage Class

Step 1: Compile and run this program **3 times**, copy the output and put it in your report, explain why the result show up that way

```
#include <stdio.h>
#include <stdlib.h>

void main()
{
    auto int x = 3;

    while (x > 0)
    {
        static int y = 5;
        y++;

        printf("The value of y is %d\n", y);
        printf("The address of y is %p\n\n", &y);

        x--;
    }
}
```

Step 2: Modify the code by delete the “static” word, compile, and re-run the program.

Step 3: Copy the output and put it in your report, explain why the results show up that way, and discuss the difference between both outputs.

Step 4: Redo step 1-3, but this time compile the program using:

“gcc -no-pie -o <your executable object name> <your c code name>”

Are they different? Why or why not

2. Extern Storage Class

Step 1: Compile and run this program **3 times**, copy the output and put it in your report, explain why the result show up that way

```
#include <stdio.h>
#include <stdlib.h>

int x = 20;

void display()
{
    extern int x;
    printf(" Display value of x: %d\n", x);
    printf(" Adress of x (in display function): %p\n\n", &x);
}

void main()
{
    extern int x;
    printf(" Print value of x: %d\n", x);
    printf(" Adress of x (in main function): %p\n\n", &x);

    display();
}
```

Step 2: Modify the code by delete the “static” word, compile, and re-run the program.

Step 3: Copy the output and put it in your report, explain why the results show up that way, and discuss the difference between both outputs.

Step 4: Redo step 1-3, but this time compile the program using:

“gcc -no-pie -o <your executable object name> <your c code name>”

Are they different? Why or why not

3. Understand the memory allocation function by following those steps.

Step 1: Run this program (without '-no-pie'):

```
#include <stdio.h>
#include <stdlib.h>

void main()
{
    int *a;
    float *b;
    int c[10];

    printf("\nAddress of Pointer\n");
    printf(">>%p\n", &a);
    printf(">>%p\n", &b);

    printf("\nEffective Address\n");
    printf(">>%p\n", a);
    printf(">>%p\n", b);

    a = (int*) malloc(10*sizeof(int));
    printf("\nAfter malloc Pointer a\n");
    printf(">>%p\n", a);
    printf(">>%p\n", &a[0]);
    printf(">>%p\n", &a[9]);

    printf("\nArray c\n");
    printf(">>%p\n", c);
    printf(">>%p\n", &c[0]);
    printf(">>%p\n", &c[9]);

    /*    b = (float*) malloc(10*sizeof(float));
    printf("\nAfter malloc Pointer b\n");
    printf(">>%p\n", b);
    printf(">>%p\n", &b[0]);
    printf(">>%p\n", &b[9]);
    */

    a = (int*) realloc(a, 1000*sizeof(int));
    printf("\nAfter realloc Pointer a\n");
    printf(">>%p\n", a);
    printf(">>%p\n", &a[0]);
    printf(">>%p\n", &a[9]);
    printf(">>%p\n", &a[999]);

    free(a);
    //    free(b);
}
```

Step 2: Copy the output and put it in your report

step 3: Explain why the results show up that way?

step 4: Modify the code by uncomment 2 places, and re-run this program

step 5: Copy the output and put it in your report

step 6: Explain why the result show up that way? And why is the result different from step 2.

Note: all steps must be executed in the same machine.

Submission

Submit the report into LEB2 before the due date. This is the problem session for the group of up to 5 students.